Basic (your first real API call) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chat_api_basics_practice.py`: this is the single call every other exercise, the Build Task, and every later document's chat feature builds on top of. Written here, once, on its own, so you see the plain request/response shape before anything else (memory, streaming, error handling) gets added on top of it. **If not:** the first time you'd see a raw `client.chat.completions.create(...)` call would be buried inside a bigger file already juggling a loop or a try/except, making it harder to tell which part is "the API call" and which part is everything else.

Solution

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def ask(system_prompt: str, user_prompt: str) -> str:
    # why: separates "how to call the API" from "which prompts to compare" —
    # comparing a third prompt is now one more call, not a copy-pasted block.
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    # how: .choices[0] is the first (and here, only) reply the model generated;
    # .message.content is the actual text, not the whole response object.
    return response.choices[0].message.content

def main() -> None:
    question = "Tell me about your day."

    formal_answer = ask("You are formal and professional.", question)
    print("Formal:", formal_answer)
```

```
    pirate_answer = ask("You are a sarcastic pirate.", question)
    print("Pirate:", pirate_answer)

if __name__ == "__main__":
    main()
```

Expected output:

```
Formal: As a formal assistant, I do not experience days the way a person does...
Pirate: Arrr, another day chained to this here API, matey...
```